

李纲
宁波大学
信息科学与
工程学院

分支界限

算法分析与设计

Analysis and Design of Computer Algorithms



- **学习要点**
- 理解分支限界法的剪枝搜索策略。
- 掌握分支限界法的算法框架
- （1）队列式(FIFO)分支限界法
- （2）优先队列式分支限界法
- 通过应用范例学习分支限界法的设计策略。
- （1）单源最短路径问题
- （2）装载问题；
- （3）布线问题
- （4）0-1背包问题；
- （5）最大团问题；
- （6）旅行售货员问题
- （7）电路板排列问题
- （8）批处理作业调度问题



6.1 分支限界法的基本思想

分支限界法与回溯法

（1）求解目标：回溯法的求解目标是找出解空间树中满足约束条件的所有解，而分支限界法的求解目标则是找出满足约束条件的一个解(?)；或是在满足约束条件的解中找出在某种意义下的最优解。

（2）搜索方式的不同：回溯法以深度优先的方式搜索解空间树，而分支限界法则以广度优先或以最小耗费优先的方式搜索解空间树。



6.1 分支限界法的基本思想

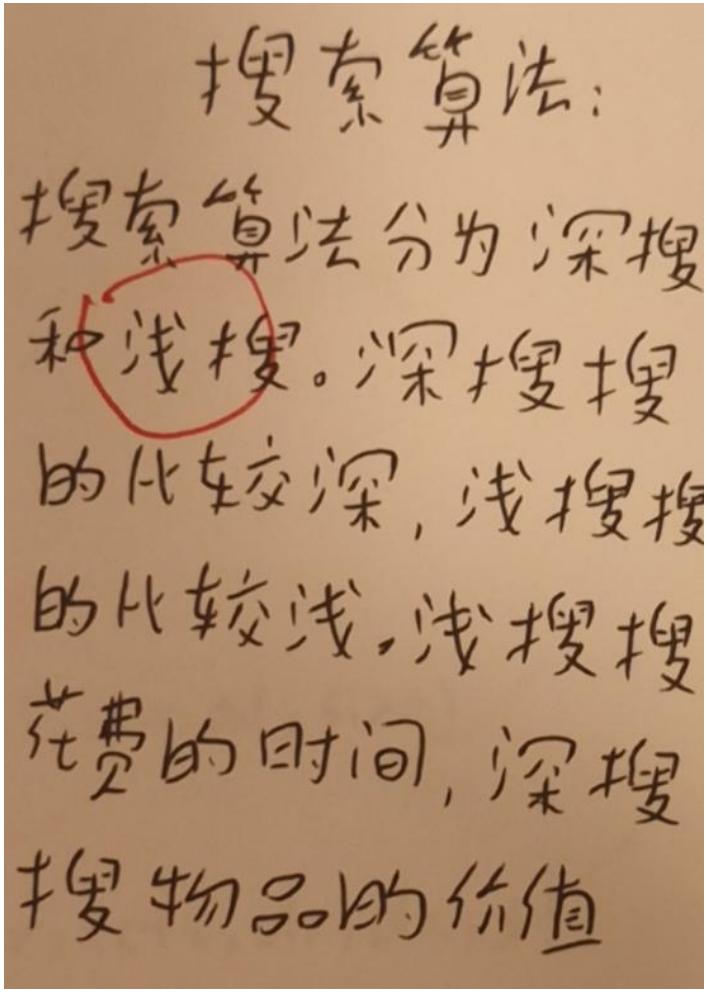
分支限界算法是按照广度优先的方式对解空间树（状态空间树）进行搜索，从而求得最优解的算法。

在搜索的过程中，采用限界函数（bound function）估算所有子节点的目标函数的可能取值，从而选择使目标函数取极值（极大值或者极小值）的节点作为扩展结点。

（如果限界值没有超过目前的最优解，则剪枝）
（重复 BFS -> 计算所有子节点限界 -> 选择最优子节点优先作为扩展结点的过程）

分支限界的思想类似于：**图的广度优先搜索，树的层序遍历。**

6.1 分支限界法的基本思想





6.1 分支限界法的基本思想

分支限界法常以广度优先或以最小耗费（最大效益）优先的方式搜索问题的解空间树。

每一个活结点只有一次机会成为扩展结点。

活结点一旦成为扩展结点，就一次性产生其所有儿子结点。

导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被加入活结点表中。

此后，从活结点表中取下一结点成为当前扩展结点，并重复上述结点扩展过程。这个过程一直持续到找到所需的解或活结点表为空时为止。



6.1 分支限界法的基本思想

常见的两种分支限界法

（1）队列式(FIFO)分支限界法

按照队列先进先出（FIFO）原则选取下一个节点为扩展节点。 P192、193①：广度

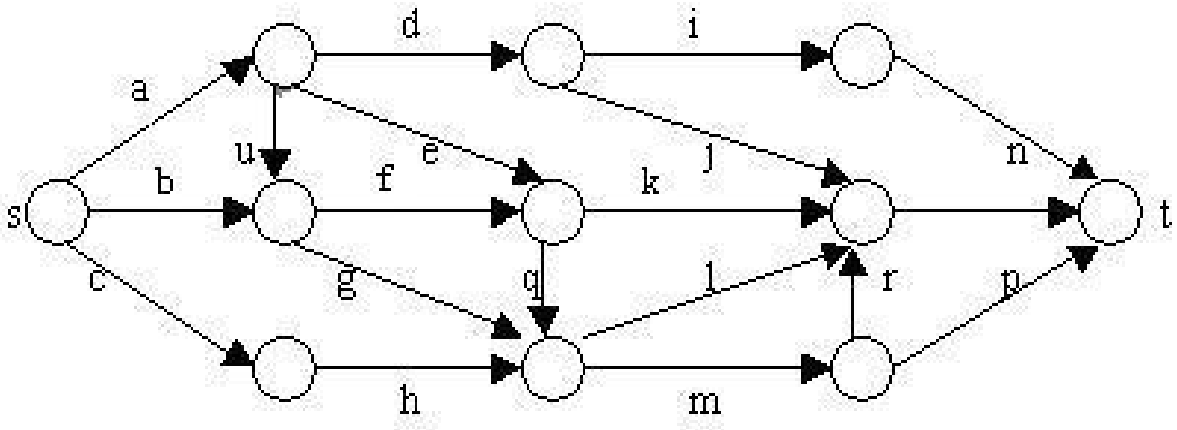
（2）优先队列式分支限界法

按照优先队列中规定的优先级选取优先级最高的节点成为当前扩展节点。P192、193②：最大堆，最小堆

6.2 单源最短路径问题

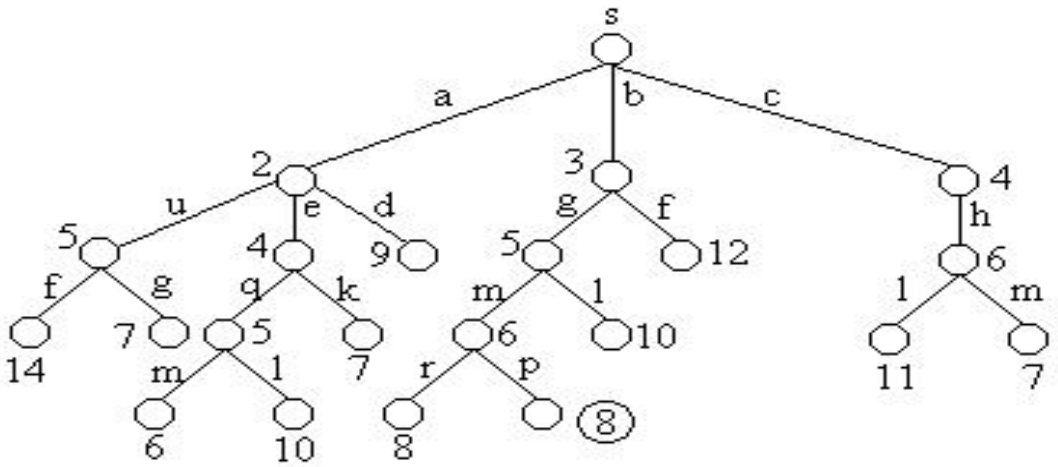
1. 问题描述

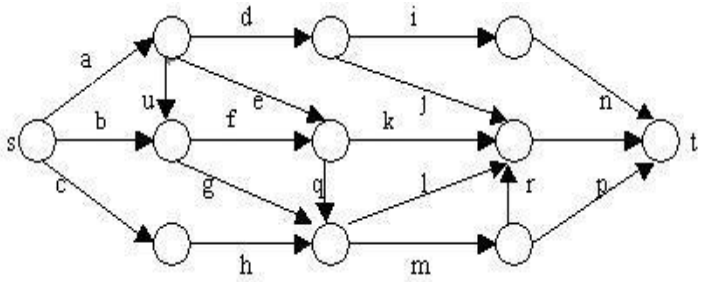
下面以一个例子来说明单源最短路径问题：在下图所给的有向图G中，每一边都有一个非负边权。要求图G的从源顶点s到目标顶点t之间的最短路径。



6.2 单源最短路径问题

下图是用优先队列式分支限界法解有向图G的单源最短路径问题产生的解空间树。其中，每一个结点旁边的数字表示该结点所对应的当前路长。（数据结构转换）







6.2 单源最短路径问题

2. 算法思想

解单源最短路径问题的优先队列式分支限界法用一极小堆来存储活结点表。其优先级是结点所对应的当前路长。

算法从图G的源顶点s和空优先队列开始。结点s被扩展后，它的儿子结点被依次插入堆中。此后，算法从堆中取出具有最小当前路长的结点作为当前扩展结点，并依次检查与当前扩展结点相邻的所有顶点。如果从当前扩展结点i到顶点j有边可达，且从源出发，途经顶点i再到顶点j的所相应的路径的长度小于当前最优路径长度，则将该顶点作为活结点插入到活结点优先队列中。这个结点的扩展过程一直继续到活结点优先队列为空时为止。

6.2 单源最短路径问题



3. 剪枝策略

在算法扩展结点的过程中，一旦发现一个结点的下界不小于当前找到的最短路长，则算法剪去以该结点为根的子树。

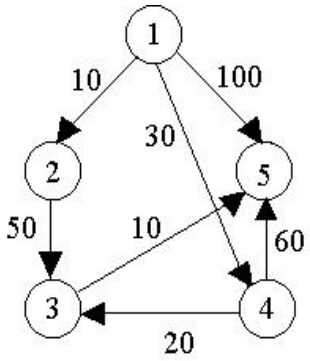
在算法中，利用结点间的控制关系进行剪枝。从源顶点s出发，2条不同路径到达图G的同一顶点。由于两条路径的路长不同，因此可以将路长长的路径所对应的树中的结点为根的子树剪去。(?)

6.2 单源最短路径问题

```
while (true) {
    for (int j = 1; j <= n; j++)
      if ((c[E.i][j]<inf)&&(E.length+c[E.i][j]<dist[j])) {
        // 顶点i到顶点j可达，且满足控制约束并比当前小
        dist[j]=E.length+c[E.i][j];
        prev[j]=E.i;
        // 加入活结点优先队列
        MinHeapNode<Type> N;
        N.i=j;
        N.length=dist[j];
        H.Insert(N);
      }
    try {H.DeleteMin(E);}        // 取下一扩展结点
    catch (OutOfBounds) {break;} // 优先队列空
    }
}
```

加入E后的变化：
单源到i的距离+i到j的距离

顶点i和j间有边，且此路径长小于原先从原点到j的路径长，更新



迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	-	10	maxint	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60



6.3 装载问题

1. 问题描述

有一批共n个集装箱要装上2艘载重量分别为C1和C2的轮船，其中集装箱i的重量为Wi，且 $\sum_{i=1}^{n} w_i \le c_1 + c_2$

装载问题要求确定是否有一个合理的装载方案可将这个集装箱装上这2艘轮船。如果有，找出一种装载方案。

容易证明：如果一个给定装载问题有解，则采用下面的策略可得到最优装载方案。

(1)首先将第一艘轮船尽可能装满；

(2)将剩余的集装箱装上第二艘轮船。



6.3 装载问题

2. 队列式分支限界法

在算法的while循环中，首先检测当前扩展结点的左儿子结点是否为可行结点。如果是则将其加入到活结点队列中。

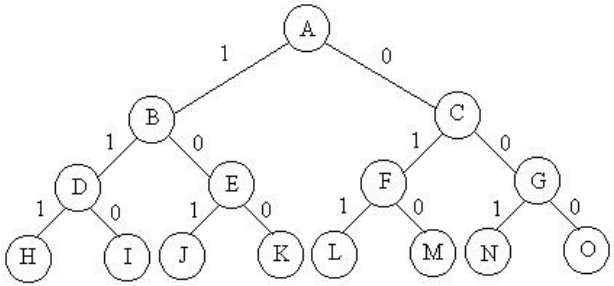
然后将其右儿子结点加入到活结点队列中(右儿子结点一定是可行结点，但不一定最优)。2个儿子结点都产生后，当前扩展结点被舍弃。

活结点队列中的队首元素被取出作为当前扩展结点，由于队列中每一层结点之后都有一个尾部标记-1，故在取队首元素时，活结点队列一定不空。当取出的元素是-1时，再判断当前队列是否为空。如果队列非空，则将尾部标记-1加入活结点队列，算法开始处理下一层的活结点。

6.3 装载问题

2. 队列式分支限界法

```
while (true) {
    // 检查左儿子结点
    if (Ew + w[i] <= c) // x[i] = 1
       EnQueue(Q, Ew + w[i], bestw, i, n);
    // 右儿子结点总是可行的
    EnQueue(Q, Ew, bestw, i, n);     // x[i] = 0
    Q.Delete(Ew);          // 取下一扩展结点
    if (Ew == -1) {        // 同层结点尾部
       if (Q.IsEmpty()) return bestw;
       Q.Add(-1);          // 新一层结点尾部标志，因为前一层扩展完了
       Q.Delete(Ew);       // 取下一扩展结点
       i++;}               // 进入下一层，+1用来记层，判别是否到叶子
}
```





6.3 装载问题

3. 算法的改进：剪枝与加速剪枝

节点的左子树表示将此集装箱装上船，右子树表示不将此集装箱装上船。设bestw是当前最优解；ew是当前扩展结点所相应的重量；r是剩余集装箱的重量。则当ew+r≤bestw时，可将其右子树剪去，因为此时若要船装最多集装箱，就应该把此箱装上船。

另外，为了确保右子树成功剪枝，应该在算法每一次进入左子树的时候更新bestw的值。



6.3 装载问题

3. 算法的改进

```
// 检查左子结点
  Type wt = Ew + w[i];
// 左儿子结点的重量
   if (wt <= c) {     // 可行结点
      if (wt > bestw) bestw = wt;
      // 加入活结点队列
      if (i < n) Q.Add(wt);
}
```

右子结点剪枝

```
// 检查右儿子结点
      if (Ew + r > bestw && i < n)
          Q.Add(Ew);  // 可能含最优解
      Q.Delete(Ew);
   // 取下一扩展结点
```

提前更新bestw
加速剪枝



6.3 装载问题

4. 构造最优解

为了在算法结束后能方便地构造出与最优值相应的最优解，算法必须存储相应子集树中从活结点到根结点的路径。为此目的，可在每个结点处设置指向其父结点的指针，并设置左、右儿子标志。

```
class QNode
{    QNode *parent;  // 指向父结点的指针
     bool LChild;        // 左儿子标志
     Type weight;        // 结点所相应的载重量
}
```



6.3 装载问题

4. 构造最优解

找到最优值后，可以根据parent回溯到根节点，找到最优解。

```
// 构造当前最优解
for (int j = n - 1; j > 0; j--) {
    bestx[j] = bestE->LChild;
    bestE = bestE->parent;
}
```



6.3 装载问题

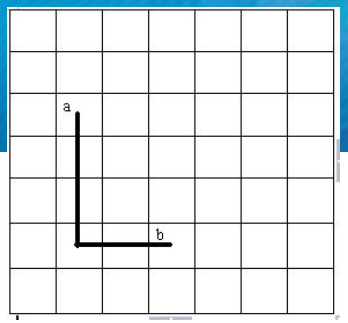
5. 优先队列式分支限界法

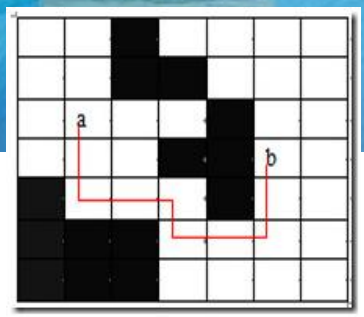
解装载问题的优先队列式分支限界法用最大优先队列存储活结点表。活结点x在优先队列中的优先级定义为从根结点到结点x的路径所相应的载重量再加上剩余集装箱的重量之和。

优先队列中优先级最大的活结点成为下一个扩展结点。以结点x为根的子树中所有结点相应的路径的载重量不超过它的优先级。子集树中叶结点所相应的载重量与其优先级相同。

在优先队列式分支限界法中，一旦有一个叶结点成为当前扩展结点，则可以断言该叶结点所相应的解即为最优解。此时可终止算法。 why?

6.4 布线问题





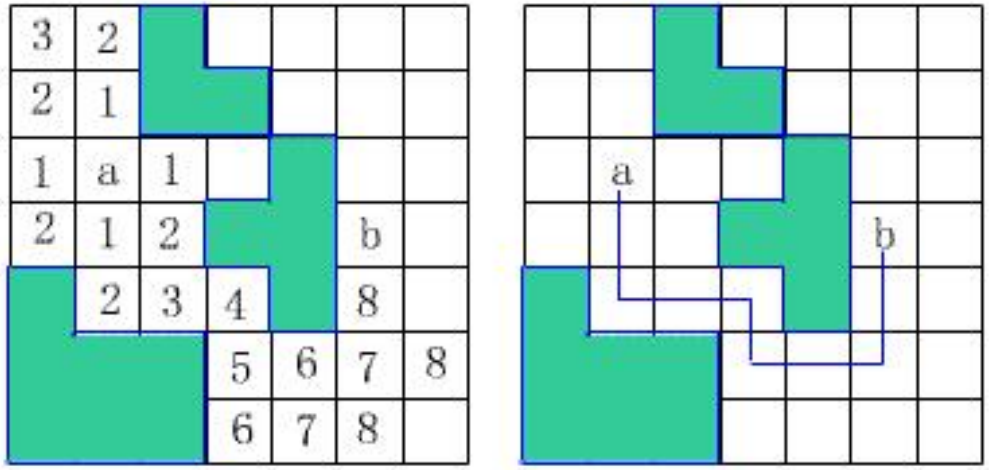
1. 算法思想

解此问题的队列式分支限界法从起始位置a开始将它作为第一个扩展结点。与该扩展结点相邻并且可达的方格成为可行结点被加入到活结点队列中，并且将这些方格标记为1，即从起始方格a到这些方格的距离为1。

P206 水漫金山

接着，算法从活结点队列中取出队首结点作为下一个扩展结点，并将与当前扩展结点相邻且未标记过的方格标记为2，并存入活结点队列。这个过程一直继续到算法搜索到目标方格b或活结点队列为空时为止。即加入剪枝的广度优先搜索。

6.4 布线问题



(a)标记距离　　(b)最短布线路径

6.4 布线问题

```
Position offset[4];
offset[0].row = 0; offset[0].col = 1; // 右
offset[1].row = 1; offset[1].col = 0; // 下
offset[2].row = 0; offset[2].col = -1; // 左
offset[3].row = -1; offset[3].col = 0; // 上
```

定义移动方向的相对位移
P204

```
for (int i = 0; i <= m+1; i++)
    grid[0][i] = grid[n+1][i] = 1; // 顶部和底部
 for (int i = 0; i <= n+1; i++)
    grid[i][0] = grid[i][m+1] = 1; // 左翼和右翼
```

设置边界的围墙



6.4 布线问题

```
for (int i = 0; i < NumOfNbrs; i++) {
        nbr.row = here.row + offset[i].row;
        nbr.col = here.col + offset[i].col;
        if (grid[nbr.row][nbr.col] == 0) {
            // 该方格未标记
            grid[nbr.row][nbr.col] = grid[here.row][here.col] + 1;
            if ((nbr.row == finish.row) &&(nbr.col == finish.col))
                break; // 完成布线
            Q.Add(nbr);}
    }
```

寻找相邻点

找到目标位置后，可以通过回溯方法找到这条最短路径。



6.5 0-1背包问题

算法的思想

首先，要对输入数据进行预处理，将各物品依其单位重量价值从大到小进行排列。

在下面描述的优先队列分支限界法中，节点的优先级由已装袋的物品价值加上剩下的最大单位重量价值的物品装满剩余容量的价值和。

算法首先检查当前扩展结点的左儿子结点的可行性。如果该左儿子结点是可行结点，则将它加入到子集树和活结点优先队列中。当前扩展结点的右儿子结点一定是可行结点，仅当右儿子结点满足上界约束时才将它加入子集树和活结点优先队列。当扩展到叶节点时为问题的最优值。



6.5 0-1背包问题

上界函数:贪心剪枝

```
while (i <= n && w[i] <= cleft)
 // n表示物品总数，cleft为剩余空间
{
        cleft -= w[i];                  //w[i]表示i所占空间
        b += p[i];                      //p[i]表示i的价值
        i++;
 }
if (i <= n) b += p[i] / w[i] * cleft; // 装填剩余容量装满背包
return b;                               //b为上界函数
```

6.5 0-1背包问题

```
while (i != n+1) {// 非叶结点
    // 检查当前扩展结点的左儿子结点
    Typew wt = cw + w[i];
    if (wt <= c) {// 左子树结点为可行结点
        if (cp+p[i] > bestp) bestp = cp+p[i];   //为什么没到叶子，就修正bestp?
        ①AddLiveNode(up, cp+p[i], cw+w[i], true, i+1);
    }
    up = Bound(i+1);
    // 检查当前扩展结点的右子树结点
    if (up >= bestp) // 右子树可能含最优解
        ②AddLiveNode(up, cp, cw, false, i+1);
    ③取下一个扩展节点（略） // 注意：不断的加入结点，而不是递归深入
}
```

分支限界搜索过程

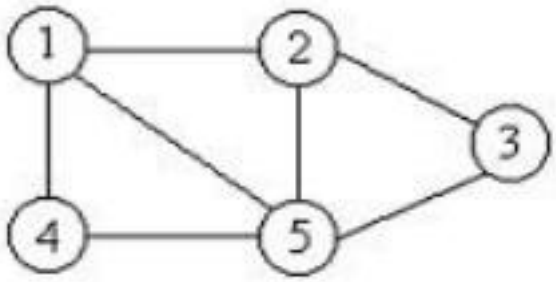


6.6 最大团问题

1. 问题描述

给定无向图G=(V，E)。如果U⊆V，且对任意u，v∈U有(u，v)∈E，则称U是G的完全子图。G的完全子图U是G的团当且仅当U不包含在G的更大的完全子图中。G的最大团是指G中所含顶点数最多的团。

下图G中，子集{1，2}是G的大小为2的完全子图。这个完全子图不是团，因为它被G的更大的完全子图{1，2，5}包含。{1，2，5}是G的最大团。{1，4，5}和{2，3，5}也是G的最大团。





2. 上界函数

用变量cliqueSize表示与该结点相应的团的顶点数；level表示结点在子集空间树中所处的层次；用cliqueSize +n-level+1作为顶点数上界upperSize的值。

在此优先队列式分支限界法中，upperSize实际上也是优先队列中元素的优先级。算法总是从活结点优先队列中抽取具有最大upperSize值的元素作为下一个扩展元素。



6.6 最大团问题

3. 算法思想

子集树的根结点是初始扩展结点，对于这个特殊的扩展结点，其cliqueSize的值为0。

算法在扩展内部结点时，首先考察其左儿子结点。在左儿子结点处，将顶点i加入到当前团中，并检查该顶点与当前团中其它顶点之间是否有边相连。当顶点i与当前团中所有顶点之间都有边相连，则相应的左儿子结点是可行结点，将它加入到子集树中并插入活结点优先队列，否则就不是可行结点。

接着继续考察当前扩展结点的右儿子结点。当upperSize>bestn时，右子树中可能含有最优解，此时将右儿子结点加入到子集树中并插入到活结点优先队列中。



6.6 最大团问题

3. 算法思想

算法的while循环的终止条件是遇到子集树中的一个叶结点(即n+1层结点)成为当前扩展结点。

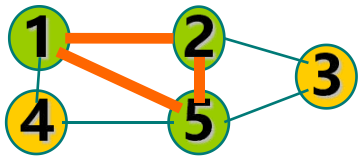
对于子集树中的**叶结点**，有upperSize = cliqueSize。此时活结点优先队列中剩余结点的upperSize值均不超过当前扩展结点的upperSize值，从而进一步搜索不可能得到更大的团，此时算法已找到一个最优解。

6.6 最大团问题

```
int Clique::BBMaxClique(int bestx[]) {
    MaxHeap<CliqueNode> H(1000);

    //初始化
    bbnode *E = 0;
    int i = 1,
    cn = 0,
    bestn = 0;

    //搜集子集空间树
    while(i!=n+1)  {   //非叶节点
       //检查顶点i与当前团中其他顶点之间是否有边相连
       bool OK = true;
       bbnode *B = E;
       for(int j=i-1; j>0; B=B->parent,j--){
          if(B->LChild && a[i][j]==0) {
             OK = false;
             break;
          }
       }
```




```
       if(OK) {  //左儿子节点为可行结点（是状态子节点）
          if(cn+1>bestn) {
             bestn = cn + 1;  //及时扩展，加快剪枝
          }
          AddLiveNode(H,cn+1,cn+n-i+1,i+1,E,true);
       }
        if(cn+n-i>=bestn){  //右子树可能含有最优解
          AddLiveNode(H,cn,cn+n-i,i+1,E,false);
       }   //注意：扩展节点而非递归深入

       //取下一扩展节点
       CliqueNode N;
       H.DeleteMax(N);   //堆非空
       E = N.ptr;
       cn = N.cn;
       i = N.level;
    }
```

```
if (OK) {// 进入左子树
   x[i] = 1;  cn++;        //加入
   Backtrack(i+1);
   x[i] = 0; cn--;}
if (cn + n - i > bestn) {// 进入右子树
   x[i] = 0;
   Backtrack(i+1);}
}
```



6.7 旅行售货员问题

1. 问题描述

某售货员要到若干城市去推销商品，已知各城市之间的路程(或旅费)。他要选定一条从驻地出发，经过每个城市一次，最后回到驻地的路线，使总的路程(或总旅费)最小。

路线是一个带权图。图中各边的费用（权）为正数。图的一条周游路线是包括V中的每个顶点在内的一条回路。周游路线的费用是这条路线上所有边的费用之和。

旅行售货员问题的解空间可以组织成一棵树，从树的根结点到任一叶结点的路径定义了图的一条周游路线。旅行售货员问题要在图G中找出费用最小的周游路线。



6.7 旅行售货员问题

这个可能性?

2. 算法描述

算法开始时创建一个最小堆，用于表示活结点优先队列。堆中每个结点的子树费用的下界lcost值是优先队列的优先级。接着算法计算出图中每个顶点的最小费用出边并用minout记录。**如果所给的有向图中某个顶点没有出边，则该图不可能有回路，算法即告结束。**如果每个顶点都有出边，则根据计算出的minout作 算法初始化。

下界：剩余顶点的最小出边之和

算法的while循环体完成对排列树内部结点的扩展。对于当前扩展结点，算法分2种情况进行处理：



6.7 旅行售货员问题

2. 算法描述

1、首先考虑s=n-2的情形，此时当前扩展结点是排列树中某个叶结点的父结点。如果该叶结点相应一条可行回路且费用小于当前最小费用，则将该叶结点插入到优先队列中，否则舍去该叶结点。 **(最后还有一个叶节点回去起点的问题)**

2、当s<n-2时，算法依次产生当前扩展结点的所有儿子结点。由于当前扩展结点所相应的路径是x[0:s]，其可行儿子结点是从剩余顶点x[s+1:n-1]中选取的顶点x[i]，且(x[s]，x[i])是所给有向图G中的一条边。对于当前扩展结点的每一个可行儿子结点，计算出其前缀(x[0:s]，x[i])的费用cc和相应的下界lcost。当lcost<bestc时，将这个可行儿子结点插入到活结点优先队列中。



6.7 旅行售货员问题

2. 算法描述

算法中while循环的终止条件是排列树的一个叶结点成为当前扩展结点。当s=n-1时，已找到的回路前缀是x[0:n-1]，它已包含图G的所有n个顶点。因此，当s=n-1时，相应的扩展结点表示一个叶结点。此时该叶结点所相应的回路的费用等于cc和lcost的值。剩余的活结点的lcost值不小于已找到的回路的费用。它们都不可能导致费用更小的回路。因此已找到的叶结点所相应的回路是一个最小费用旅行售货员回路，算法可以结束。

算法结束时返回找到的最小费用，相应的最优解由数组v给出。



6.9 批处理作业调度问题

1. 问题的描述

给定n个作业的集合$J=\{J_1,J_2,\ldots,J_n\}$。每一个作业J_i都有2项任务要分别在2台机器上完成。每一个作业必须先由机器1处理，然后再由机器2处理。作业J_i需要机器j的处理时间为t_{ji}，i=1,2,...,n；j=1,2。对于一个确定的作业调度，设是F_{ji}是作业i在机器j上完成处理的时间。则所有作业在机器2上完成处理的时间和

$$f = \sum_{i=1}^{n} F_{2i}$$

称为该作业调度的完成时间和。批处理作业调度问题要求对于给定的n个作业，制定最佳作业调度方案，使其完成时间和达到最小。



6.9 批处理作业调度问题

2. 限界函数

在结点E处相应子树中叶结点完成时间和的**下界是**:

$$f \geq \sum_{i \in M} F_{2i} + \max\{S_1, S_2\}$$

s1、s2两种最省时间的可能
P220
s1:机器1没有空闲剩余作业的时间
s2:机器2没有空闲剩余作业的时间

注意到如果选择P_k，使t_{1pk}在k>=r+1时依非减序排列，S1则取得极小值。同理如果选择P_k使t_{2pk}依非减序排列，则S2取得极小值。

$$f \geq \sum_{i \in M} F_{2i} + \max\{\hat{S}_1, \hat{S}_2\}$$

这可以作为优先队列式分支限界法中的限界函数。

5 4 3 2 1

5 9 12 14 15

1 2 3 4 5

1 3 6 10 15



6.9 批处理作业调度问题

2. 限界函数

设|M|＝r，且L是以结点e为根的子树中的一个叶结点，相应的作业调度为{pk , k=1,2,...,n}，其中pk是第k个安排的作业。如果从结点E开始到叶结点L的路上，每一个作业pk在机器1上完成处理后都能立即在机器2上开始处理，即从pr＋1开始机器1没有空闲时间，则对于该叶结点L有：

$$\sum F2i \geq \sum[F1pr + (n-k+1)\ t1pk + t2pk] = S1 \quad (i \notin M)$$

类似地，如果从结点E开始到结点L的路上，从作业pr＋1开始，机器2没有空闲时间，则：

$$\sum F2i \geq \sum[\max(F2pr , F1pr + \min t1i) + (n-k+1)t2pk] = S2 \quad (i \notin M)$$



6.9 批处理作业调度问题

3. 算法描述

算法的while循环完成对排列树内部结点的有序扩展。在while循环体内算法依次从活结点优先队列中取出具有最小bb值（完成时间和下界）的结点作为当前扩展结点，并加以扩展。

（1）首先考虑**E.s=n**的情形，当前扩展结点E是排列树中的叶结点。E.sf2是相应于该叶结点的完成时间和。当E.sf2 < bestc时更新当前最优值bestc和相应的当前最优解bestx。

（2）当**E.s<n**时，算法依次产生当前扩展结点E的所有儿子结点。对于当前扩展结点的每一个儿子结点node，计算出其相应的完成时间和的下界bb。当bb < bestc时，将该儿子结点插入到活结点优先队列中。而当bb≥ bestc时，可将结点node舍去。



6.9 批处理作业调度问题

3. 算法描述

```
while (E.s <= n ) {
 if (E.s == n ) { // 叶结点
   if (E.sf2 < bestc) {
       bestc = E.sf2;
       for (int i = 0; i < n; i++)   bestx[i] =E.x[i];
   }
   delete [] E.x;}
```

当E.sf2<bestc时，更新当前最优值bestc和相应的最优解bestx



6.9 批处理作业调度问题

3. 算法描述

```
else {// 产生当前扩展结点的儿子结点
      for (int i = E.s; i < n; i++) {
          Swap(E.x[E.s],E.x[i]);
          int f1,f2;
          int bb=Bound(E,f1,f2,y);
          if (bb < bestc ) {
             MinHeapNode N;
             N.NewNode(E,f1,f2,bb,n);
             H.Insert(N);}
          Swap(E.x[E.s],E.x[i]);
          }
      delete [] E.x;}  // 完成结点扩展
```

当bb<bestc时，将儿子结点插入到活结点优先队列中



6.10 讨论

给定两个正整数A和B，它们不超过1e6，且不含数字0。

同时给定两种编辑方式：

1、删除某个位置的数字；

2、在某个位置添加一个不为0的数字。

试设计一个分支限界算法，将正整数A转变为正整数B，且使编辑次数最少。算法输入为两个不超过1e6的整数A和B，输出最少的编辑次数。

最直接思路：

1、有多少种操作/状态

2、每次展开所有操作/状态；入栈



6.10 讨论

给定两个正整数A和B，它们不超过1e6，且不含数字0。同时给定两种编辑方式：删除某个位置的数字；在某个位置添加一个不为0的数字。试设计一个分支限界算法，将正整数A转变为正整数B，且使编辑次数最少。算法输入为两个不超过1e6的整数A和B，输出最少的编辑次数。

```
Q.push((A,0));
while (Q非空) {
  取出队列头部元素（S,e）；    //e表示操作次数
  if（s==B） return e;
  for  (i=1; i<=S的长度;++i) {
    删除S的第i位数字，得到整数T;   Q.push((T,e+1));
  }
  for  (i=0; i<=S的长度;++i) {
     for (j=1; j<10; ++j) {
         在S的第i位之后，插入数字j，得到整数T;  Q.push((T,e+1));
     }
  }
}
```



6.10 讨论

给定两个正整数A和B，它们不超过1e6，且不含数字0。同时给定两种编辑方式：删除某个位置的数字；在某个位置添加一个不为0的数字。试设计一个分支限界算法，将正整数A转变为正整数B，且使编辑次数最少。算法输入为两个不超过1e6的整数A和B，输出最少的编辑次数。

方法2：先求字长公共子序列

[len(A)-lcs(A,B)] + [len(B)-lcs(A,B)]

方法3：

```
matchstr(A,B,i,j){
        if (A结束或B结束) {...}
        if A[i]==B[j]  matchstr(A,B,i+1,j+1);
        else, 有两个分支：
          1:delete A[i]：matchstr(A,B,i+1,j); 恢复状态;
          2:insert B[j]：  matchstr(A,B,i,j+1); 恢复状态;
        //或者也可以认为分支2是deleteB[j]
}
```